

apollo

APOLLO 2035

全国无人驾驶出行运营中枢

参赛赛道 AI+ 工业软件赛道

企业命题 百度地图赛题：基于百度地图 JSAPI 的海量空间数据流式高性能渲染

代码仓库 github.com/aeryking/apollo-2035 · Apache-2.0

800 万辆	60 FPS	91 ms	104 万段
实时仿真 · 每秒更新	平移 / 缩放 / 旋转 / 倾斜	首辆车上屏	8 城真实路网

01 · 项目背景

一张地图，实时看清全国几百万辆车

赛题要解决的问题

工业场景里的空间数据有三个共同特点：**规模大**（百万级点、线、面）、**持续更新**（设备、车辆每秒上报）、**需要边看边操作**（运维人员频繁平移、缩放、旋转地图）。传统做法是先整包下载、再一次性绘制，数据一大就要长时间等待，交互时卡顿明显。

我们选的载体

设想 2035 年，全国无人驾驶网约车全面普及。运营中枢需要在一张地图上看清：**车在哪、在做什么、下一刻城市哪里需要车、急救车怎样穿城而过**。这正是「海量空间数据 + 持续更新 + 高频交互」的典型场景。

车辆、需求与事件均为程序生成的模拟数据；「Apollo」「2035」为概念演示设定，不代表百度官方数据或产品。

与赛题列举的工业场景一一对应

本项目	对应的工业场景
100 万～800 万辆车，每秒上报位置（点）	设备点位监控、人员定位
104 万段路网，沿程流光（线）	物流干线、管网运行态势
行政区划、需求热度 3D 柱（面）	能源消耗分布、园区分区
城市潮汐、网约车需求预测	大规模车辆调度
120 / 119 生命通道，沿途车辆让行	园区安防与应急调度

02 · 核心成果

在一台 8 GB 笔记本上的实测结果

800 万辆

实时仿真，59.4 FPS

每辆车每秒更新一次位置（快照 0.96 Hz），每秒向显存上传约 95 MB 新数据

60 FPS

交互全程满帧

100 万 / 400 万辆，平移、缩放、旋转、倾斜、城市近景 57.8–60.6 FPS，交互期 0 长任务

91 ms

首辆车上屏

首条路网 77 ms（Safari，100 万辆）；Chrome 186 / 199 ms。数据边下载边显示

6.2 ms

单帧中位数

关闭垂直同步后 100 万辆约 154 FPS，60 FPS 是屏幕刷新率上限，不是方案上限

2.2×

密度自适应 LOD

200 万辆全国视角：26.9 → 60 FPS，车辆图层 GPU 耗时 67 ms → 7.2 ms

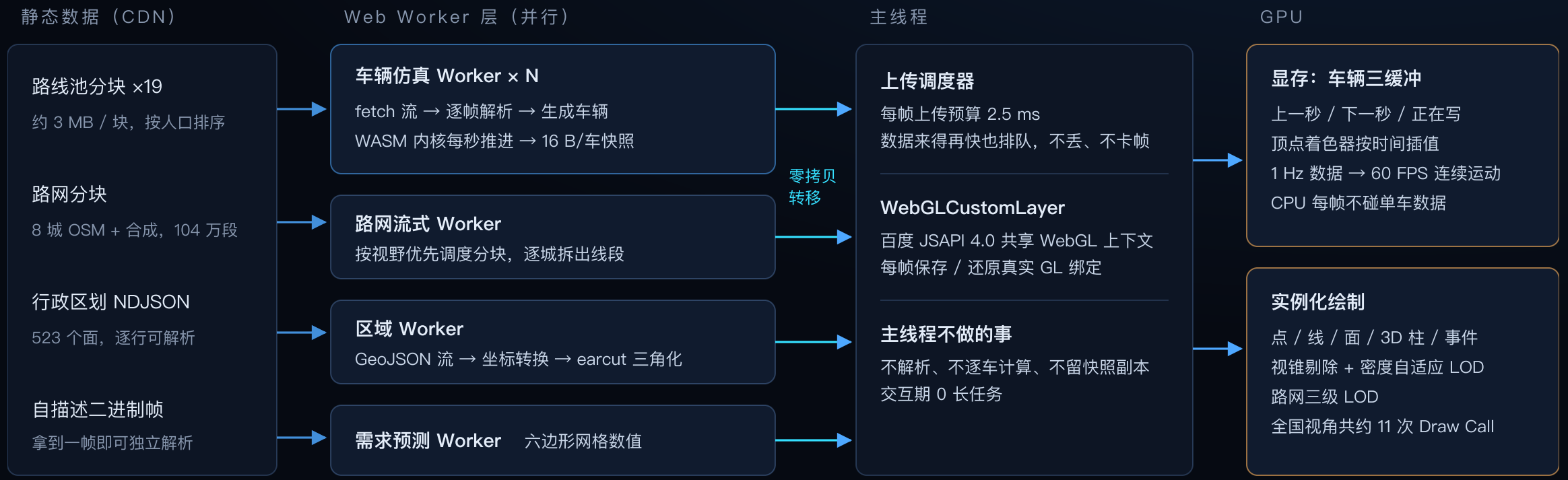
4.2%

加载期本项目主线程占比

解析、计算全部在 Worker 中完成；主线程只负责按帧预算上传显存和发出绘制命令

测试设备：MacBook Air（Apple M2，8 GB 统一内存，集成显卡，60 Hz 屏），Chrome 154 / Safari 27，生产构建，接通电源，普通浏览器（非自动化）。首屏与加载时间在 6 城真实路网（38.7 MB）版本上测得，当前版本为 8 城含支路（55.8 MB）。完整数据见仓库 docs/性能测试报告.md。

从网络到显卡，全程不卡主线程



「获取 → 解析 → 计算 → 上传 → 绘制」五段全部异步化，彼此用可转移对象衔接。主线程只做两件事：按帧时间预算把数据写进显存，发出绘制命令。

04 · 海量空间数据流式处理模块

边下载、边解析、边显示

自描述二进制帧协议

- 文件头 32 字节 + 若干「类型 · 长度 · 载荷」帧，拿到一帧就能独立解析，不必等整个文件下载完。
- 解析器支持任意切分的字节流，半包留到下一块，逐字节喂入的单元测试通过。
- 几何在构建期已投影成图层坐标并打包成 GPU 可直接使用的结构，解析就是取视图，不产生中间对象。

分块与调度

- 375 座城市合并为 19 个约 3 MB 的区域块：逐城 490 个小请求实测只有 1.4 MB/s，合并后单流读取，下载与解析流水线重叠。
- 块内按城市人口降序，最重要的数据最先到达。
- **视野优先**：未开始的块按「离视野中心的距离 ÷ 城市重要度」排序，地图移动后重排，用户看哪里，哪里先出数据。
- 按数据块贪心装箱，把负载均衡分给各仿真 Worker。

增量上传：时间预算 + 三缓冲

- 每帧上传不超过 2.5 ms，按 16K 条切块，每块后检查预算，数据再多也不会让单帧超时。
- 车辆快照三缓冲轮换：写入的永远是约 1 秒前已不再被 GPU 读取的那一块，不撕裂、不触发 CPU 等 GPU。
- N 个 Worker 的 1 Hz 节拍错开 1/N 秒，上传负载在一秒内均匀分布。

「流式」在画面上看得见

- 城市数据到达的那一刻，对应城市的光晕闪光点亮：全国 375 座城市按人口逐座亮起。
- 左下角实时显示已接入数据量和加载吞吐；首辆车 91 ms 上屏，加载期本项目代码只占主线程 4.2%。
- WASM 仿真内核（Rust，2 KB）比同逻辑 JS 快 1.74 倍（浏览器内）/ 3.3 倍（Node），为更大规模留出 CPU。

05 · WebGL 高性能渲染模块

百万级要素，个位数 Draw Call

要素	规模	渲染方式
车辆（点）	100 万~800 万	远景点精灵，近景实例化车身；GPU 快照插值；视锥剔除；密度自适应 LOD
路网（线）	104 万段	实例化线段，像素恒宽；着色器流光；按道路等级三级 LOD
区域（面）	523 面 / 48.9 万顶点	Worker 内 earcut 三角化，增量写入
需求预测（3D 柱）	2,568 柱	共享六棱柱模板，数值双缓冲，高度在着色器中插值
城市光晕 / 城际流线	375 / 约 1 万段	各 1 次实例化绘制，流动完全由时间驱动
特种车辆	救护车、消防车、警车	贴图图集，随行驶方向旋转，警灯红蓝交替

关键手段

- **GPU 插值**：仿真每秒一次，画面每秒 60 次，CPU 不参与逐帧计算。
- **16 字节紧凑记录**：100 万辆三缓冲只占 46 MB 显存。
- **密度自适应 LOD**：全国视角一个像素上叠着成千上万辆车，按屏幕密度每 k 辆取 1 辆。
- **无分支点着色器**：去掉 discard 与 exp，在分块渲染 GPU 上 51.6 → 60.1 FPS。

200 万辆，全国视角	LOD 关	LOD 开
Chrome FPS	26.9	**60**
车辆图层 GPU 耗时	67.0 ms	**7.2 ms**
Safari FPS	28.5	**61.1**

06 · 百度 WebGLLayer 深度适配

官方文档没写、但决定成败的几件事

① MVP 矩阵只在着色器用到时才注入

占位着色器若没有引用 `u_mvpMatrix`，`getMVPMatrix()` 恒为全零，所有顶点塌缩到一点，画面空白且没有任何报错。对策：显式引用该矩阵。

② 安全模式不还原真实 GL 绑定

底图渲染器依赖状态缓存跳过重复绑定。改了缓冲、程序、VAO 不还原，下一帧底图会读到我们的数据，整屏空白。对策：每帧前后保存、还原真实绑定。

③ 代理 GL 上绑定的纹理读出全黑

图层拿到的是带缓存的代理对象，经它绑定的纹理在绘制时采样为黑色。特种车辆图标改用真实上下文绘制，绘制后逐项还原状态，底图不受影响。

④ 坐标系与旋转方向

百度墨卡托是分纬度带多项式，不是标准 Web Mercator。我们从官方运行时移植了正反两套系数，Worker 里不依赖 BMap 也能算出逐像素对齐的坐标，实测四种交互下偏差小于 0.5 像素；地图旋转以逆时针为正。

全部为本机实测结论，记录在仓库 docs/技术方案.md 与 docs/00-平台实测.md。另外实测发现：百度地图会识别自动化浏览器并降级（空地图只有 0.5–3 FPS），因此全部性能数据都在普通浏览器中采集。

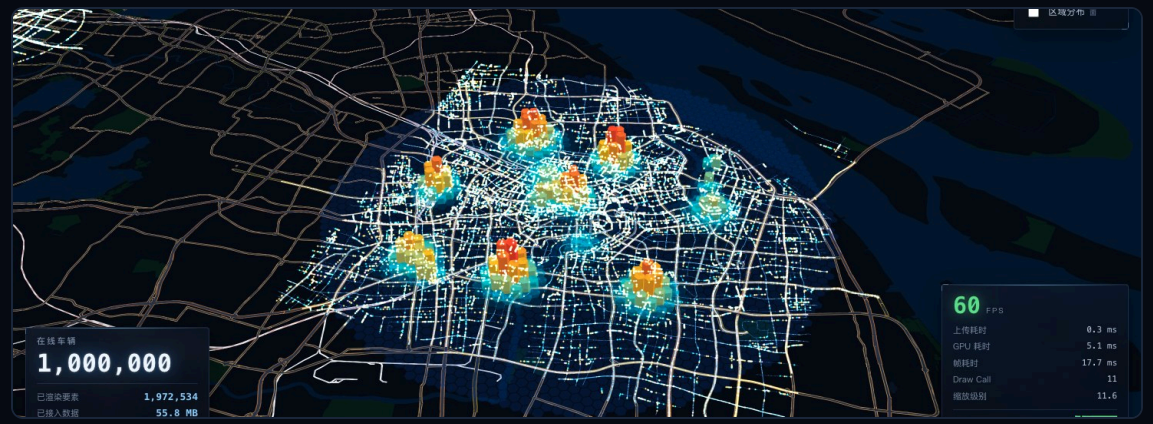
四幕演示：从全国到街道



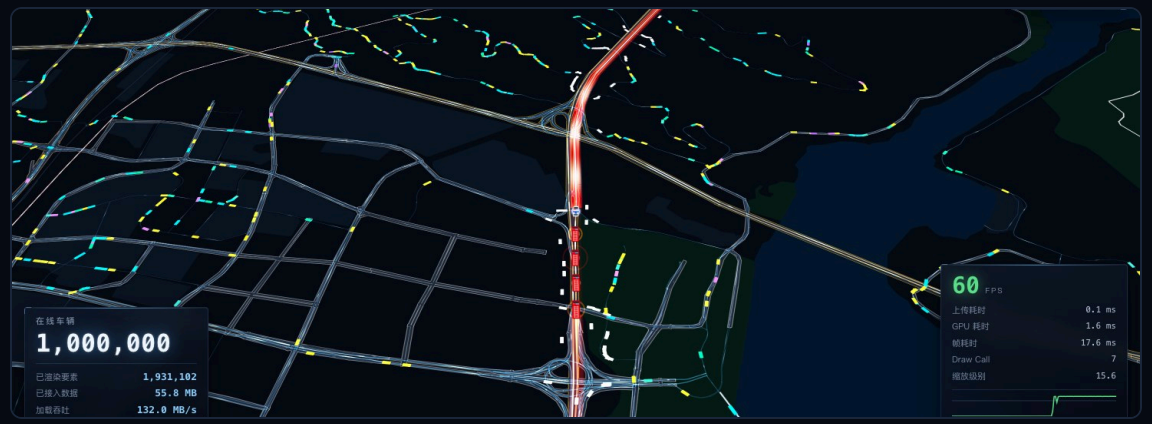
I 全国脉搏 100 万辆车在线，375 座城市随数据到达逐座点亮，城际流线串起城市群



II 城市潮汐 1 秒 = 3 分钟，早高峰载客车流涌向市中心；8 座真实路网城市可选



III 网约车需求预测 3D 柱表示 15 分钟后的叫车需求，柱子越高需求越大，空车提前调过去



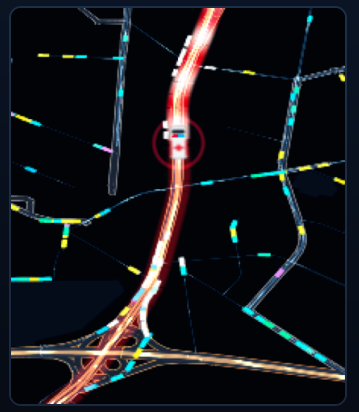
IV 生命通道 119 消防车队由警车开道穿城，无人机视角追车，沿途车辆自动靠边让行

应急调度：120 救护车与 119 消防车队



120 生命通道 · 北京　镜头从救护车后上方追拍，前方路段车辆变白、靠边停下





- 8 座真实路网城市都能开通，城市和事件可在二级菜单选择或随机。
- 路线由仿真 Worker 自动挑选：要求顺直、穿城，并按沿线实际车流密度排序。
- Worker 只查询车队前方 1.6～2.2 km 内的车辆，让行判断不扫全城。
- 警车、消防车、救护车为 WebGL 贴图，不是网页元素，不影响百万辆车的帧率。

规模从 100 万到 800 万

完整测试 · Chrome · 100 万辆

阶段	平均 FPS	1% Low	P95 帧时	长任务
数据加载中	36.9	4.4	137.4 ms	3
稳态	60	57.1	17.0 ms	0
平移	60	56.8	17.2 ms	0
缩放	57.8	43.7	20.9 ms	0
旋转	60	56.8	17.2 ms	0
倾斜	60	57.1	17.1 ms	0
城市近景	60	56.8	17.3 ms	0

数据加载约 1 秒，期间主线程开销主要来自百度地图引擎初始化（53.3%），本项目代码占 4.2%。

实时仿真规模与数据新鲜度

车辆	浏览器	平均 FPS	快照频率	每 Worker CPU
100 万	Chrome	60	1 Hz	1.2%
200 万	Chrome	60	0.98 Hz	2.5%
400 万	Chrome	60	0.94 Hz	3.2%
600 万	Safari	60	0.98 Hz	4.7%
800 万	Chrome	**59.4**	**0.96 Hz**	6.6%

只看帧率不够：上传跟不上时帧率依旧 60，车辆位置却会变旧。所以同时记录每秒快照换页次数（理想 1 Hz）。显存：100 万辆 124 MB，400 万辆 261 MB。

和常规做法相比

环节	常规做法	本方案
数据加载	整包下载 JSON，解析完才显示	自描述二进制帧流式解析，视野优先调度，首辆车 91 ms 上屏
数据处理	在主线程解析、计算	全部在 Worker 中完成，零拷贝交给主线程；仿真内核为 WASM
车辆运动	CPU 每帧更新每辆车的位置	每秒一次快照，GPU 在两份快照之间插值，CPU 每帧不碰单车
显存更新	数据一到就整块重传，容易卡帧	每帧 2.5 ms 上传预算 + 三缓冲轮换 + Worker 节拍错开
海量点的绘制	DOM 标注或逐点绘制，万级开始卡顿	实例化绘制 + 视锥剔除 + 屏幕密度自适应 LOD，800 万辆约 60 FPS
性能验证	只报一个 FPS 数字	普通浏览器实测（规避百度对自动化环境的降级），同时记录 1% Low、长任务、GPU 耗时、数据新鲜度，脚本可复现

一句话：**在一台 8 GB 笔记本上，用百度地图 WebGLLayer 实时渲染 800 万辆每秒更新的车辆，同时叠加 104 万段路网和全国行政区划，交互全程约 60 FPS。**

代码、数据、测试都可以复现

赛题交付物对照

赛题要求	仓库中的位置
基于 JSAPI WebGLLayer 的 Web Demo	全部图层基于 `BMap.WebGLCustomLayer`，`src/layers/`
海量空间数据流式处理模块	`src/stream/`、`src/sim/`、`wasm/sim/`、`tools/build-*.mjs`
WebGL 高性能渲染模块	`src/layers/`、`src/core/gl.js`
性能测试报告	`docs/性能测试报告.md`，测试工具 `src/bench/`、`tools/bench/`
技术文档	`docs/技术方案.md`、`docs/00-平台实测.md`

合规

- 代码以 **Apache-2.0** 开源，第三方署名见 NOTICE。
- 路网来自 OpenStreetMap（ODbL），行政区划来自 DataV.GeoAtlas，均已注明出处。
- 百度 AK 通过 `.env` 配置，不入库，仓库只提供 `.env.example`。
- 运行时依赖只有 earcut（ISC），构建依赖 Vite（MIT）。

一条命令复现

- `npm install && npm run dev` 运行演示；`?debug=1` 自动演示四幕。
- `?bench=1&fleet=N` 运行内置性能测试，完成后显示结果并可下载 JSON / Markdown。
- `npm run data:fetch / data:build` 从原始数据重建全部数据块。

11 · 长期发展

从概念演示到真实业务

近期　接入实时数据

- 数据源从浏览器内仿真换成 WebSocket 推送，渲染端不用改。
- 按视野订阅，只推屏幕范围内的车；远景只推城市聚合值。
- 位置改用 16 位相对坐标和增量编码，把每个访客的流量降到每秒几十到几百 KB。

中期　全国全量路网

- 路网从按城市分块改为矢量瓦片，按视野和缩放级别加载，覆盖全国全部等级道路。
- 车辆快照量化为 int16，上传带宽再减半。
- 在 Windows、独立显卡和 120 Hz 屏幕上补测，测试页和结果回收端可直接在局域网使用。

长期　通用化与社区

- 把流式帧协议、上传调度器、GPU 插值图层抽成独立的 WebGLLayer 组件库，可用于设备监控、物流轨迹、管网态势等场景。
- 探索 WebGPU 计算着色器做大规模空间查询。
- 完善贡献指南、Issue 模板与持续集成，按版本发布。

代码仓库	许可证	版本
github.com/aeryking/apollo-2035	Apache-2.0	v1.0.0 · 2026-10-16